# Ellis Architecture

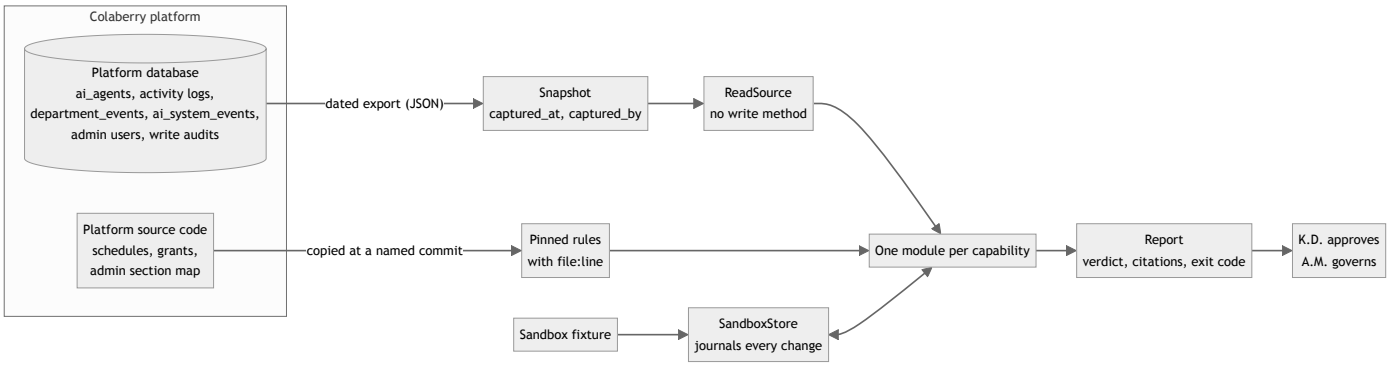
Ellis watches the AI agents that run the Colaberry platform and tells a person, with evidence, what is healthy, what is broken, and what it cannot see. This page shows how it is built and why each part is shaped the way it is.

| 36 of 48 | 566 | 0 | 0 |
|---|---|---|---|
| capabilities built and verified | tests, all passing | runtime dependencies | writes to the live platform |

SYSTEM CONTEXT

## Ellis reads the platform, never writes to it

The platform is a production system with more than a hundred scheduled agents. Ellis takes two things from it: a dated export of the records it needs, and the rules it needs from the source code, copied at a named commit. Every capability reads through an interface that has no write method. The only thing Ellis ever changes is a sandbox copy.




The export is pulled by a person with admin access. Ellis holds no database credentials and has no network path to the platform.

LAYERS

## Five layers, each with one job

L1

**Input: dated snapshots**

Each capability takes a JSON export with a capture time and who captured it. A snapshot missing a required table is refused, not read as empty.

`src/readSource.ts`
`src/access/accessSnapshot.ts`
`examples/*.json`

L2

### Read boundary

Capabilities see the platform only through `ReadSource`, which can find and list rows but has no way to change them. Errors are classified as `SourceUnavailable` or `ContractViolation`.

`src/readSource.ts`
`src/records.ts`

---

L3

### Pinned platform knowledge

Rules Ellis needs from the platform's code are copied with the commit and file:line they came from: agent grants, the admin section map, the scanner list and the retention sweeps. Each copy names its source, so anyone can check it against the platform, and a test pins the copied values so they cannot change unnoticed.

`src/grants/agentGrants.ts`
`src/access/sectionMap.ts`
`src/retention/sweepCatalogue.ts`

---

L4

### Capabilities

One module per capability, each a pure function from snapshot to report. Same rows in any order give the same report, and most suites have a test for it.

`src/capabilities/*.ts`
`src/sandbox/sandboxStore.ts`

---

L5

### Output: reports a person can check

Every verdict carries citations, the table and row id it came from. Each command exits with a code per state, so a scheduler or a person can act on it without parsing text.

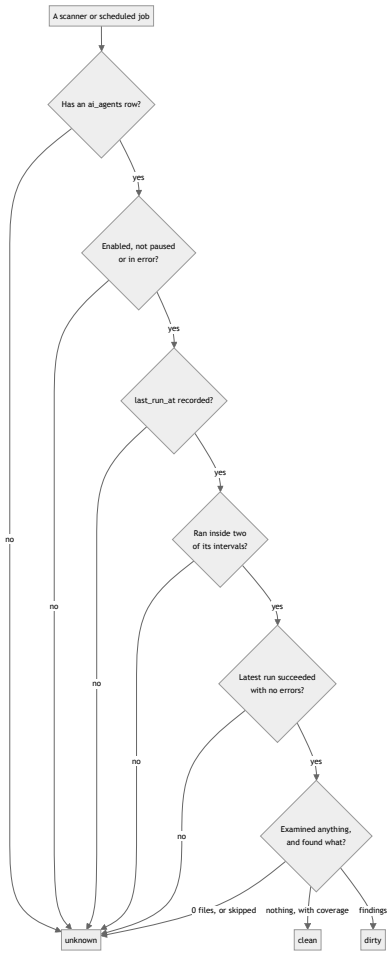
`src/cli/*.ts`
`npm run <capability>`

---

THE CORE RULE

## Unknown is checked before clean

The platform's own monitors repeatedly call a silent scanner healthy. Ellis asks every reason a source could be silent before it looks at any result, so "nothing found" and "nothing looked" can never be confused.




● clean ● dirty ● unknown

This is the rule in the security rollup and the security posture scan. The data retention sweep applies the same idea to retention windows: a window that is only in code is unknown, never a number of days.

CAPABILITY REGISTER

# What Ellis can answer today

| # | CAPABILITY | ANSWER STATES | EXIT CODES | AUTONOMY NOW → CEILING |
|---|---|---|---|---|
| 1 | **Orchestration health watch**<br>Is orchestration healthy right now, and when did that last change? | ● healthy ● degraded ● critical ● run_failed ● stalled ● not_running ● unknown | 0 answer · 2 refused | R2 → R5 |
| 2 | **Orchestration auto-repair**<br>Do the two approved repairs work on a sandbox, changing each fault once however often they run? | ● changed ● no_op ● partial ● failed | 0 verified · 1 failed | R4 → R6 |
| 3 | **Security posture scan**<br>What did the last secret, code and dependency scan find, and when? | ● findings ● clean ● unknown | 0 known · 3 unknown | R2 → R5 |

| # | CAPABILITY | ANSWER STATES | EXIT CODES | AUTONOMY NOW → CEILING |
|---|---|---|---|---|
| 4 | **Runtime threat and access watch**<br>Who can reach which admin section, and what changed since the last check? | no admin row · row, no sections · bridge only · has sections | 0 answer | R2 → R5 |
| 5 | **Agent behaviour and AI-safety watch**<br>Which agents acted outside their grants, and which grant was missing? | finding · unattributable · not emitted | 0 none · 4 findings | R2 → R6 |
| 6 | **Security oversight rollup**<br>Is the platform safe right now, as one verdict over all seven scanners? | green · red · unknown | 0 · 3 unknown · 4 red | R2 → R5 |
| 7 | **Data retention sweeps**<br>What is each table's retention window, and when was it last swept? | stated · conflicting · unknown | 0 all known · 3 | R2 → R4 |
| 8 | **Preview stack reaping**<br>Which preview stacks are live, how idle is each, and which are reap candidates? | active · reap candidate · unknown idle | 0 all known · 3 unknown or attention | R2 → R4 |
| 9 | **Stale action recovery**<br>What is stuck right now, for how long, and on which attempt? | ok · stuck · past attempt cap · unknown | 0 none · 3 stuck or unknown · 4 past cap | R2 → R4 |
| 10 | **Agent ticket auto-resolution**<br>Do the ticket auto-closers close only finished work, and do nothing on a second run? (sandbox) | verified · not verified | 0 verified · 3 not | R4 → R7 |
| 11 | **Infrastructure evolution proposals**<br>What has been proposed, why, and what happened to it? | accepted · deferred · rejected · open | 0 all decided · 3 any open | R2 → R5 |
| 12 | **System resilience supervision**<br>How resilient is the platform right now, and how old is each part's last signal? | healthy · degraded · unknown | 0 healthy · 3 unknown · 4 degraded | R2 → R5 |
| 13 | **Agent enable, disable and pause control**<br>Should this agent be stopped, and would pause or disable actually stop it? | disable recommended · no recommendation · pause holds unless a run fails · no row | 0 recommended · 3 none | R2 → R2 |
| 14 | **Workforce technology direction**<br>What tooling does the fleet hold, and where does it diverge from the written direction? | no divergence · divergent · unknown | 0 none · 3 unknown · 4 divergent | R2 → R5 |
| 15 | **Prompt and performance optimisation**<br>Which prompts cost the most, which are slowest, and which have open change proposals? | measured · partial · unknown | 0 measured · 3 partial or unknown | R2 → R5 |
| 16 | **Platform bug fix dispatch**<br>For a reported bug, does a sandboxed baseline test fail for the reported reason before any fix? | baseline confirmed · passes before fix · unreproducible · other reason · did not run | 0 confirmed · 4 unreproducible · 3 not | R1 → R4 |
| 17 | **Agent registry read surface**<br>Is this agent registered, is it on, when does it really run, and what has it done? | registered · disabled · no recorded runs · not registered | 0 registered · 3 not or ambiguous | R2 → R2 |
| 18 | **Cron run instrumentation**<br>Did this job run, when, how long did it take, and did it fail? | run recorded · no run recorded · not instrumented | 0 run recorded · 3 any not | R2 → R2 |

| # | CAPABILITY | ANSWER STATES | EXIT CODES | AUTONOMY NOW → CEILING |
|---|---|---|---|---|
| 19 | **Cron health alerting**<br>Which cron jobs are error-spiking or overdue right now, and which cannot be evaluated at all? | alerting · clear · evaluated in part · not evaluated | 0 clear · 4 alerting · 3 blind spots | R2 → R5 |
| 20 | **Agent health scoring**<br>Which agents are worst off, and which scores rest on zero runs? | scored · unscored · score cannot fall · no row | 0 all scored · 4 failures behind no score · 3 unscored | R2 → R4 |
| 21 | **Timer and agent registry reconciliation**<br>Which timers are untracked, which registry rows are phantom, and which schedules disagree, measured and dated? | matches baseline · new drift · healed entries · no baseline | 0 matches · 4 new drift · 3 healed only | R2 → R4 |
| 22 | **Incident trace and error triage**<br>What run caused this failure, and where no trace exists, why not? | traced · not traceable · not found · no failure recorded | 0 traced · 3 not traceable · 4 nothing found | R2 → R5 |
| 23 | **Schedule governance and live reload**<br>Which schedule change is worth proposing, and can it be applied live or only with a restart? | reloadable · inline, restart only · both · home unknown | 0 known home · 3 unknown home · 4 nothing to propose | R2 → R2 |
| 24 | **Dead-letter capture**<br>Which jobs reached the dead-letter store, and which jobs can never reach it? | dead lettered · queues empty | 0 dead lettered · 4 empty | R2 → R4 |
| 25 | **Dead-letter triage**<br>What is in the dead-letter queue, what should a person do with each entry, and what could not have reached it? | queue open · queue empty | 0 open · 4 empty | R2 → R4 |
| 26 | **Error classification coverage**<br>What share of recorded failures carry a specific class, and which cron catch sites record none? | all classified · gaps · no failures | 0 classified · 3 gaps · 4 none | R2 → R4 |
| 27 | **Idempotency and replay audit**<br>Can a sandbox run tell a job that is safe to run twice from one that is not? | harness proven · cannot fail | 0 proven · 3 cannot fail | R1 → R4 |
| 28 | **Cron overlap guard**<br>Which timers can run longer than their own interval, and which jobs hold their own claim? | none seen · can overlap · unassessable | 0 none · 3 can overlap · 4 unassessable | R2 → R4 |
| 29 | **Schema drift detection**<br>What did the latest drift run report, and what does it never compare? | none reported · drift reported · unknown | 0 none · 3 drift · 4 unknown | R2 → R3 |
| 30 | **Backup and restore verification**<br>Can anyone state that a current backup exists and can be restored, and what search says so? | lines to read · unknown | 3 lines to read · 4 unknown · 5 search incomplete | R2 → R3 |
| 31 | **Deploy verification evidence**<br>Do the deployed commit, the running image and the live routes agree for this release? | evidence agrees · disagrees · unverified | 0 agrees · 3 disagrees · 4 unverified | R2 → R2 |
| 32 | **Production deploy review and approval**<br>Is the checkout clean and level with the remote, before a person decides to deploy? | ready · not ready · remote unread | 0 ready · 3 not ready · 4 remote unread | R1 → R1 |

| # | CAPABILITY | ANSWER STATES | EXIT CODES | AUTONOMY NOW → CEILING |
|---|---|---|---|---|
| 33 | **Reliability briefing for the accountable manager**<br>What ran, what failed, what can nobody measure, and what needs a decision? | ● records read ● run figures unknown | 0 records read · 4 figures unknown | R2 → R5 |
| 34 | **Reliability threshold ownership**<br>Which numbers decide when the platform complains, where do they live, and what are they producing? | ● all measured ● unmeasured ● inventory unread | 0 measured · 4 unmeasured · 5 unread | R2 → R5 |
| 35 | **Agent promotion and approval decision**<br>Does this capability meet this rung's criteria, and on what evidence? | ● meets criteria ● unproven or unmet ● rung out of reach | 0 meets · 4 unproven or unmet · 5 out of reach | R2 → R2 |
| 36 | **Per-capability autonomy authority resolution**<br>What is this capability's effective authority today, and which constraint is binding? | ● resolved ● no authority ● rules unconfirmed | 0 resolved · 4 no authority · 5 unconfirmed | R2 → R2 |

Every command also exits 1 for bad input. Autonomy starts at R0 for each capability; the first release reads and reports (R2), except auto-repair and ticket auto-resolution, whose first releases run multi-step tasks on a sandbox (R4), and three at R1: bug fix dispatch, which only prepares a baseline test, the idempotency audit, which runs fixture jobs in memory, and production deploy review, which reports facts for a person. Promotion is one step at a time, on evidence, with a named person's approval: K.D. to R2, A.M. from R3.

SAFETY BOUNDARIES

# What Ellis is built to be unable to do

### Change the live platform

Capabilities receive a `ReadSource`. There is no write method to call.

`src/readSource.ts`

### Repair anything real

Repairs run only on a `SandboxStore`, which journals every change and refuses writes that change nothing. There is no production target in the code.

`src/sandbox/sandboxStore.ts`

### Hold credentials

An access snapshot carrying password, token or secret columns is refused before it is read.

`src/access/accessSnapshot.ts`

### Repeat a leaked secret

Security findings pass through a field whitelist. A scanner's stored snippet of the matching line never leaves the module.

`src/capabilities/securityPostureScan.ts`

### Police itself

The behaviour watch excludes the watchers' own actions, Ellis included, and counts them separately.

`src/capabilities/agentBehaviorSafetyWatch.ts`

### Let a test reach out

Baseline tests run under Node's permission model: they read and write only their own sandbox folder, start no other programs and open no network. The platform checkout is only ever read.

`src/sandbox/baselineRunner.ts`

### Stop or start an agent

Ellis says what a disable or pause would and would not stop. It has no call that changes an agent's state; a person does that.

`src/capabilities/agentEnableDisableControl.ts`

### Make a person's decision

Ellis proposes a schedule change or what to do with a dead letter, with the evidence. Applying a change, replaying, abandoning and marking resolved stay with a person at every level, and replay is always labelled unproven.

`src/capabilities/deadLetterTriage.ts`

### Guess

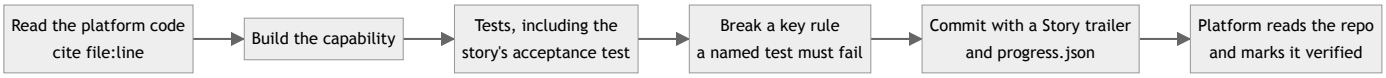
A missing row, run, count or window is reported as unknown or not recorded, with the reason. It is never filled in. A job with no record is "not instrumented", never "did not run", and no alert is never read as healthy: a job the alert check cannot see is listed as not evaluated.

`every capability`

VERIFICATION

## How a capability is proven

Each capability starts from reading the platform code it depends on, and ends with the platform itself confirming the work from this repo. Defects are for Ellis to find and report in its own output, by K.D. and A.M.'s decision of 30 Oct.




The break step is a manual mutation test: change the rule in the code, run the suite, confirm which named test fails, then restore the code. Each capability's write-up records which test caught each break.

ROAD TO PRODUCTION

## Dev, then test, then production

NOW · DEV

### Proven on tests and example exports

All twenty-five capabilities pass their tests and run on example inputs; the bug baseline has also been proven on real platform code. None has been run on live records yet.

NEXT · TEST

### First live check

One export from A.M.'s admin screen, with Ellis's answers checked against it together with K.D. K.D. cannot open the admin surfaces today (`requireAdmin`, `aiOpsRoutes.ts:44`).

THEN · PRODUCTION

### Scheduled, still read-only

Ellis runs on a schedule against fresh exports. Any action beyond reporting moves up the autonomy ladder one step at a time, with K.D.'s sign-off.

STACK

## Small on purpose

TypeScript run natively by Node, with the standard library's test runner. No framework and no runtime packages, so there is nothing to patch and it can all be read in a day: 93 source files, about 10,000 lines, and 566 tests.

`TypeScript 5.8, strict` `Node 23.6+ type stripping` `node:test` `tsc --noEmit` `JSON snapshots` `0 runtime dependencies`

---

Repository: [github.com/qninying/platform-reliability-agent](github.com/qninying/platform-reliability-agent)

Platform facts are pinned to the public Colaberry platform repository at the commit named in each capability's write-up, from `c860274` to `bbde6ef`. State as of 2026-11-02: 36 capabilities verified.